

Designing and Analysis of Algorithms

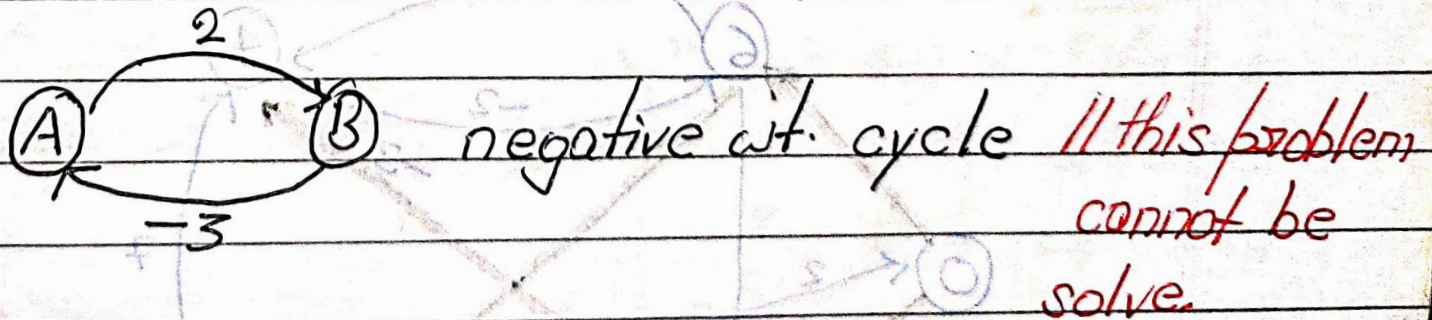
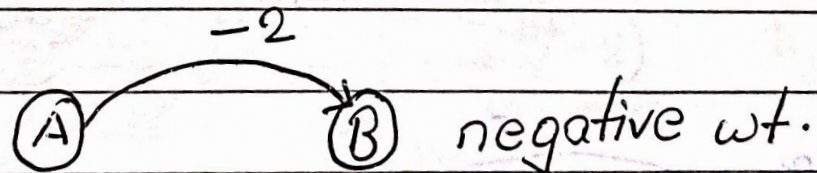
Course Code: ECS 5101/CS514

- Dr Rahul Mishra
- IIT Patna

- **Lecture 25**

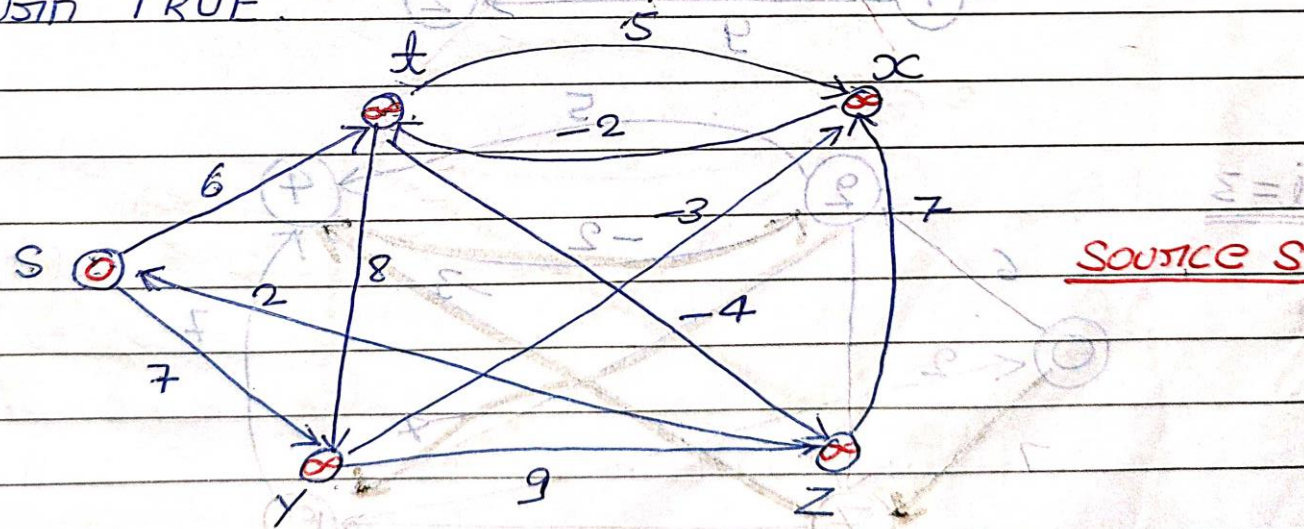
Bellman-Ford Algorithm:

The Bellman Ford Algorithm solve the single source shortest path problem in general case in which edge weight may negative. It return boolean value which is either true / false. It Return true in solvable problem i.e; there is no negative wt cycle otherwise false.



Bellman-Ford (G, w, s)

1. Initialize-Single-Source (G, s)
2. for $i \leftarrow 1$ to $|V[G]| - 1$
3. do for each edge $(u, v) \in E[G]$
4. do Relax (u, v)
5. for each edge $(u, v) \in E[G]$
6. do if $d[v] > d[u] + w(u, v)$
7. then return FALSE
8. return TRUE.

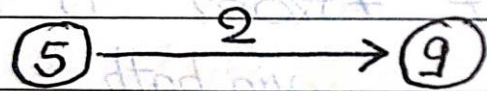


• **Relax** (u, v, w)

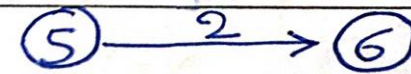
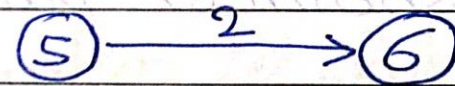
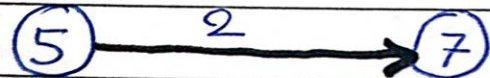
1. if $d[v] > d[u] + w(u, v)$

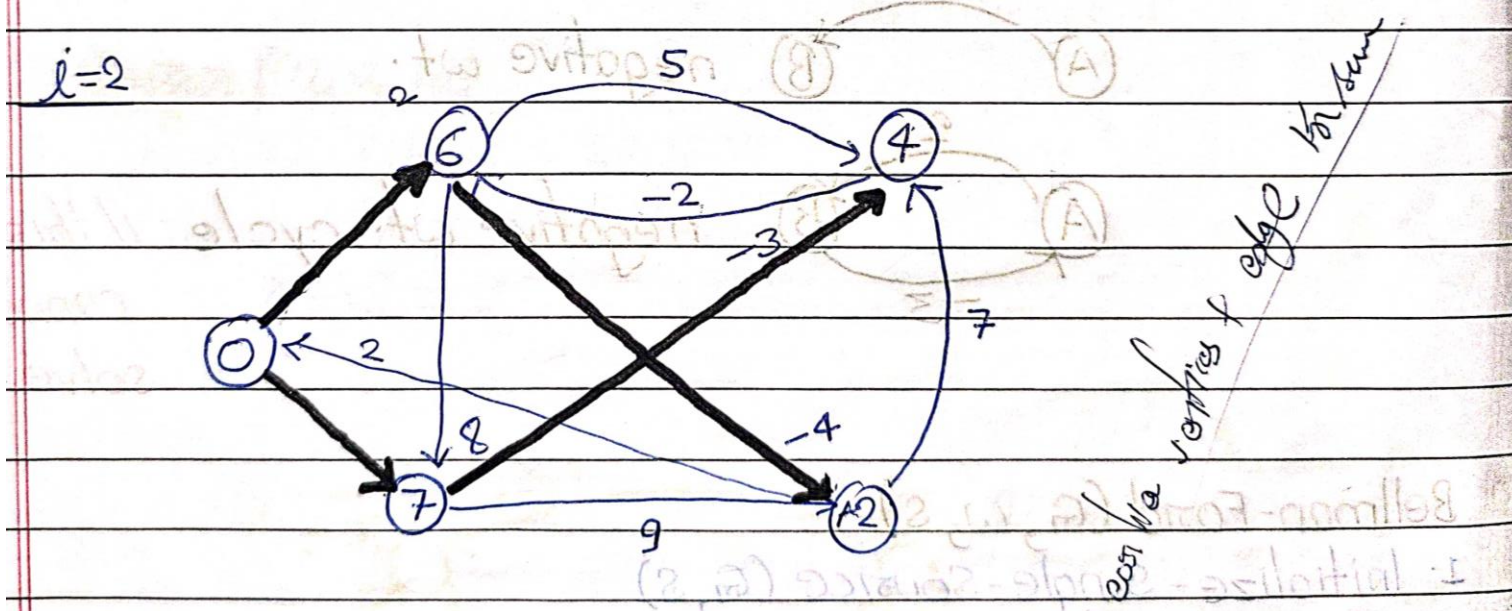
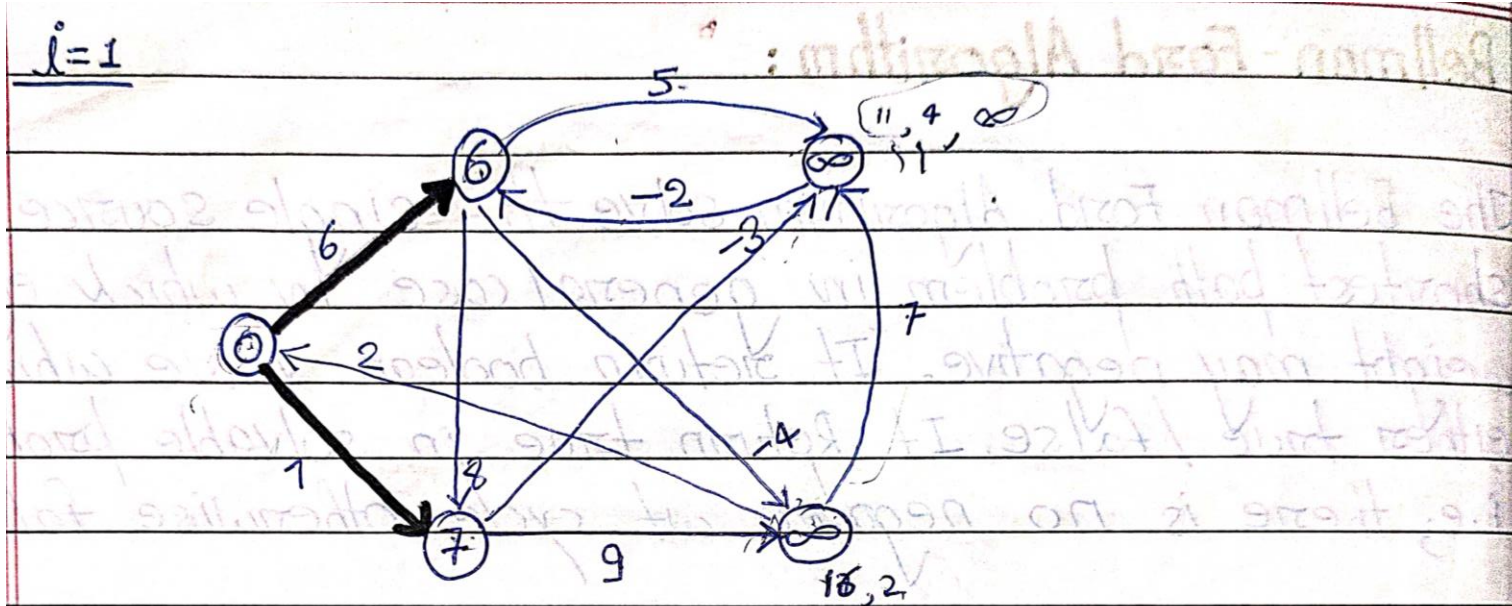
then $d[v] \leftarrow d[u] + w(u, v)$

$\pi[v] \leftarrow u$

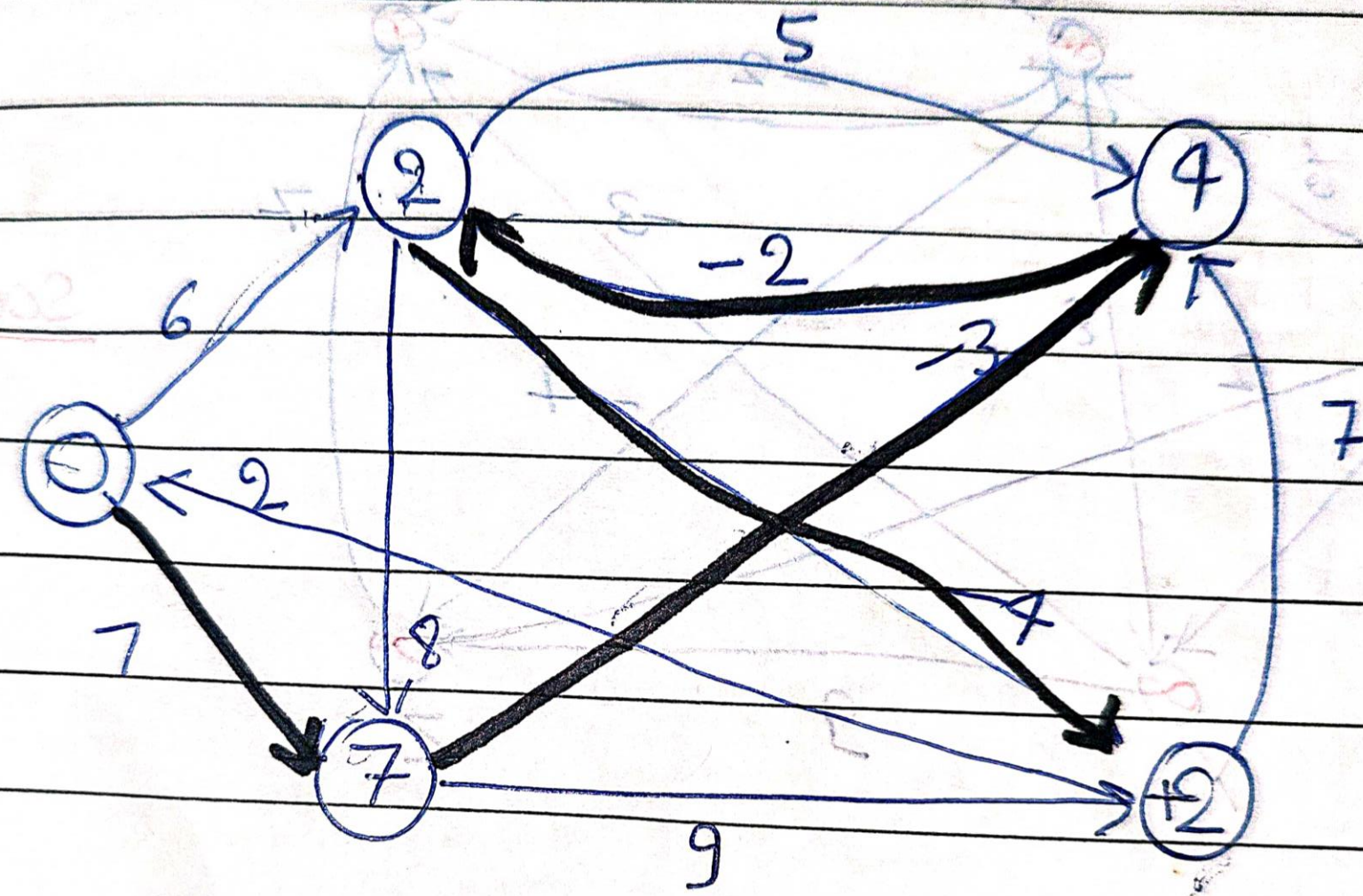


Relax



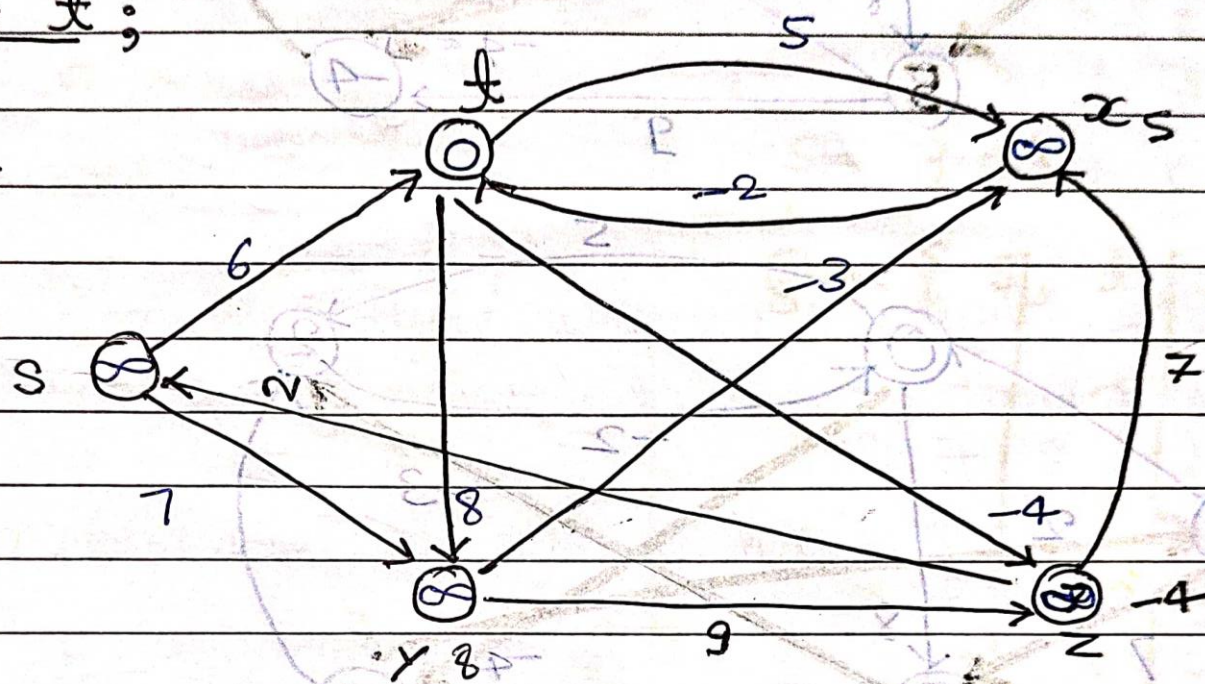


$i=3$

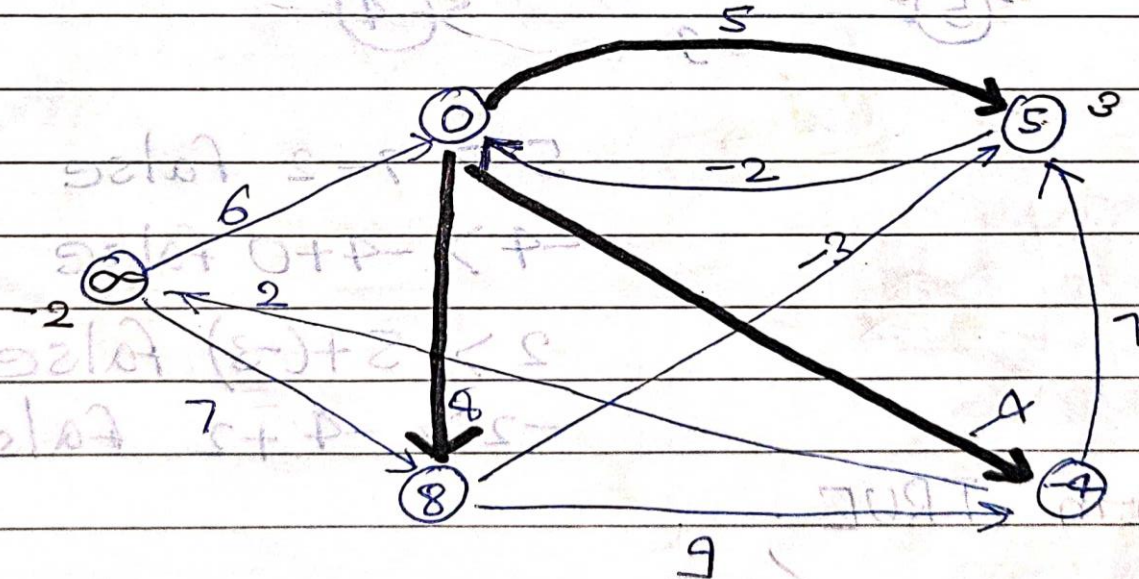


Source t;

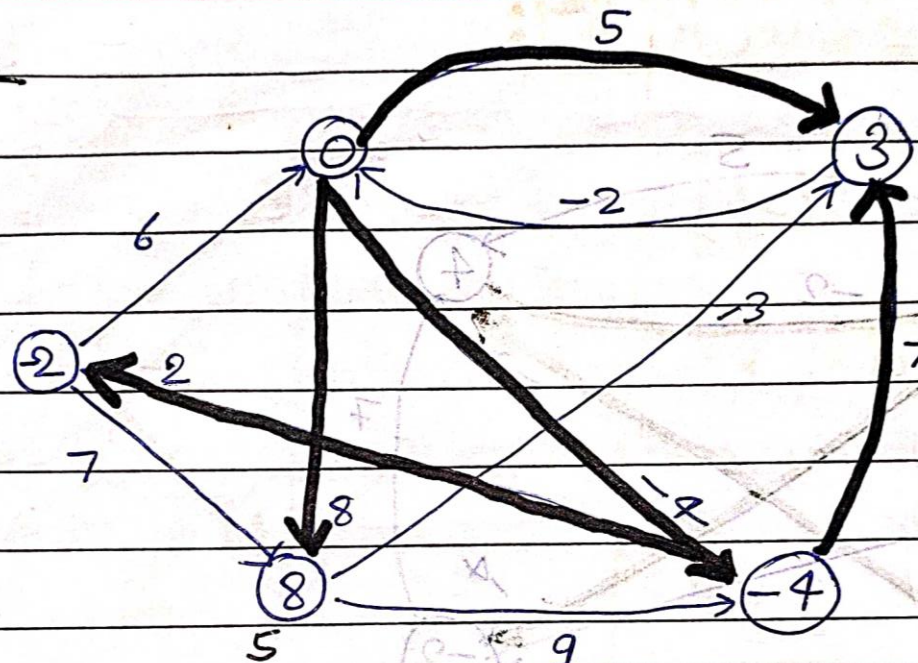
Ques:-



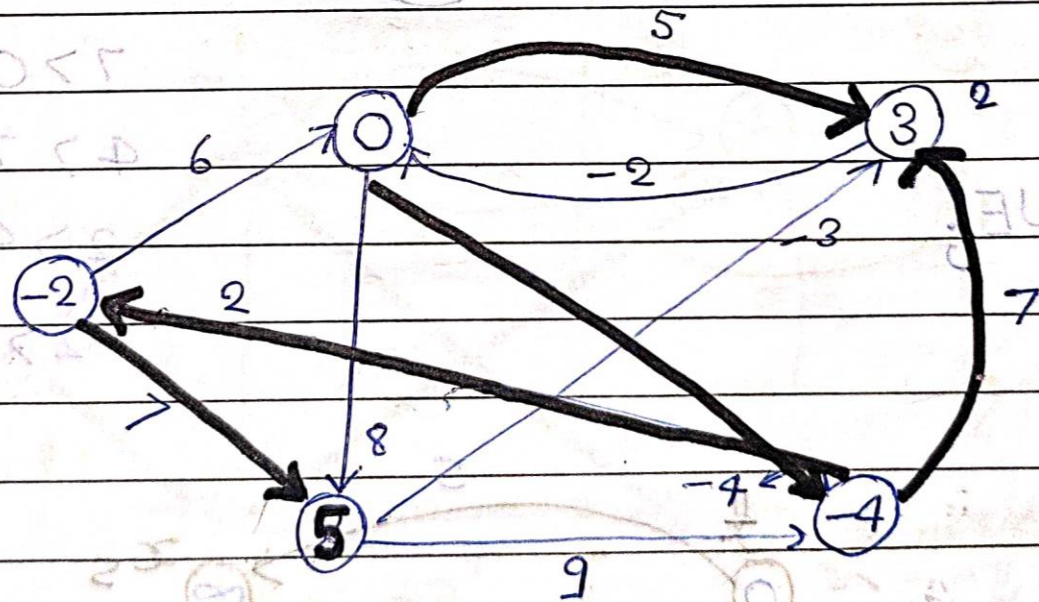
i=1



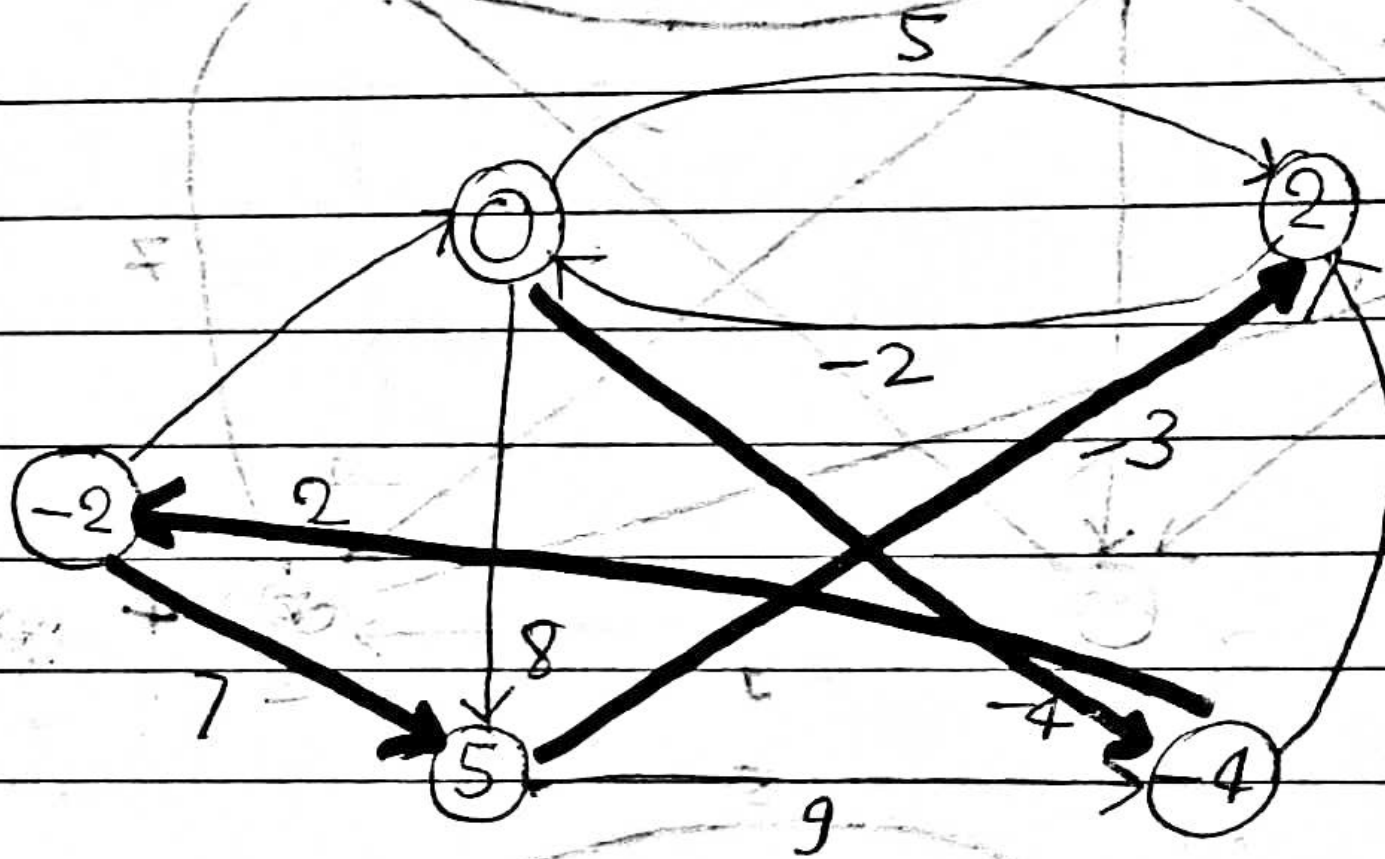
$i=2$



$i=3$



$i=4$



$5 > 7 - 2$ false

DIJKSTRA ALGORITHM:

Dijkstra algorithm solves the single source shortest path problem on a weighted directed graph from the case in which all edge weight are non-negative.

Running time $\Rightarrow O(E \log V)$

DIJKSTRA (G, w, s)

1. Initialize single source (G, s)

2. $S \leftarrow \emptyset$

3. $Q \leftarrow V[G]$

4. while $Q \neq \emptyset$

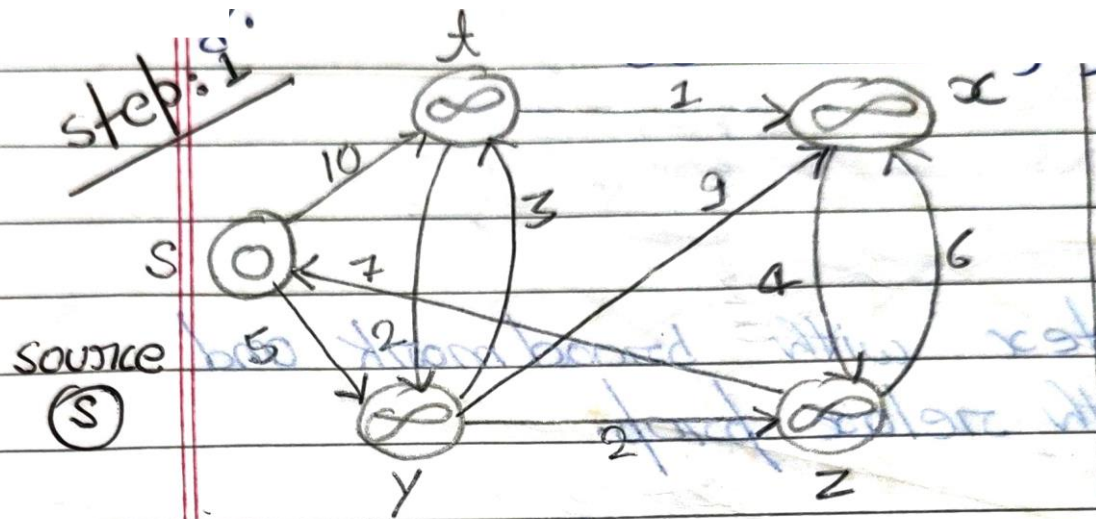
5. do $U \leftarrow \text{Extract_Min}(Q)$

6. $S \leftarrow S \cup \{U\}$

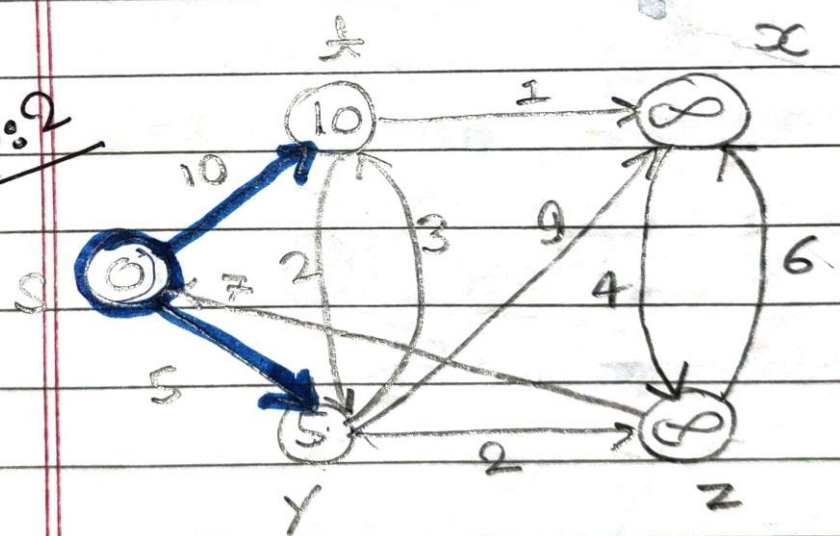
7. for each vertex $v \in \text{Adj}[U]$

8. do Relax (U, v, w)

step:1



step:2

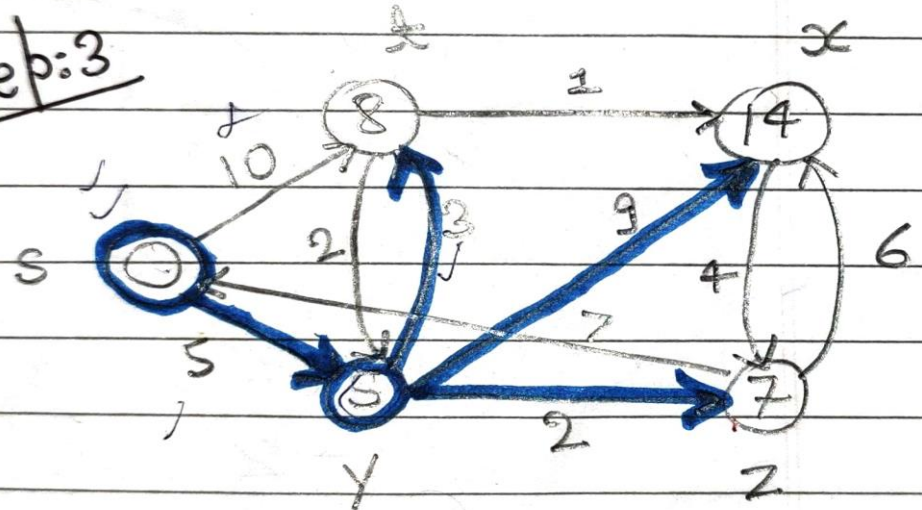


$$S = \{\phi\}$$

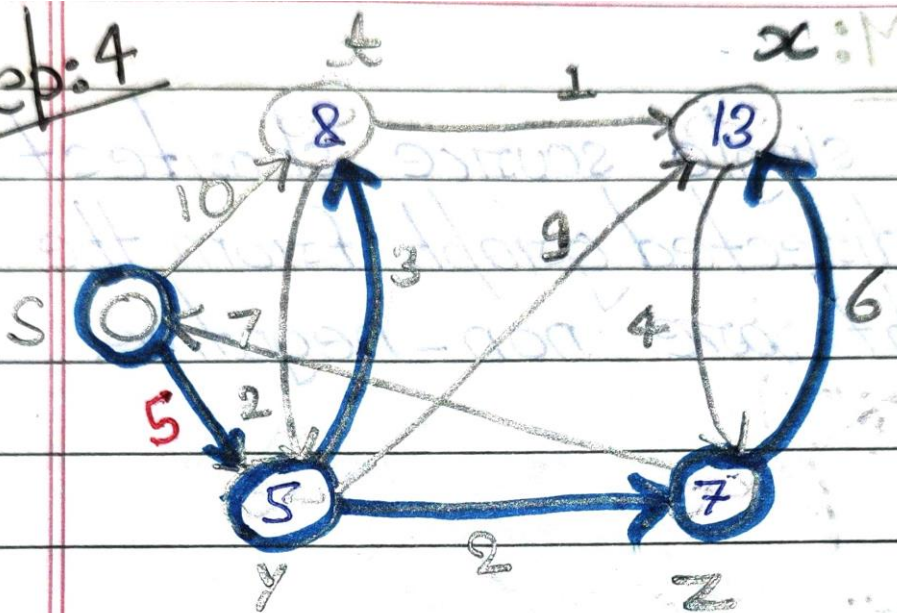
$$Q \leftarrow \{s, t, x, y, z\}$$

	s	t	x	y	z	
-	∞	∞	∞	∞	∞	$\Rightarrow S = \{s\}$
-	10	∞	∞	5	∞	$\Rightarrow S = \{s, y\}$
-	8	14	-	-	7	$\Rightarrow S = \{s, z, y\}$
-	8	13	-	-	-	$\Rightarrow S = \{s, z, y, t\}$
-	-	9	-	-	-	$\Rightarrow S = \{s, z, y, t, x\}$
-	-	-	-	-	-	$\Rightarrow S = \{s, y, z, t, x\}$

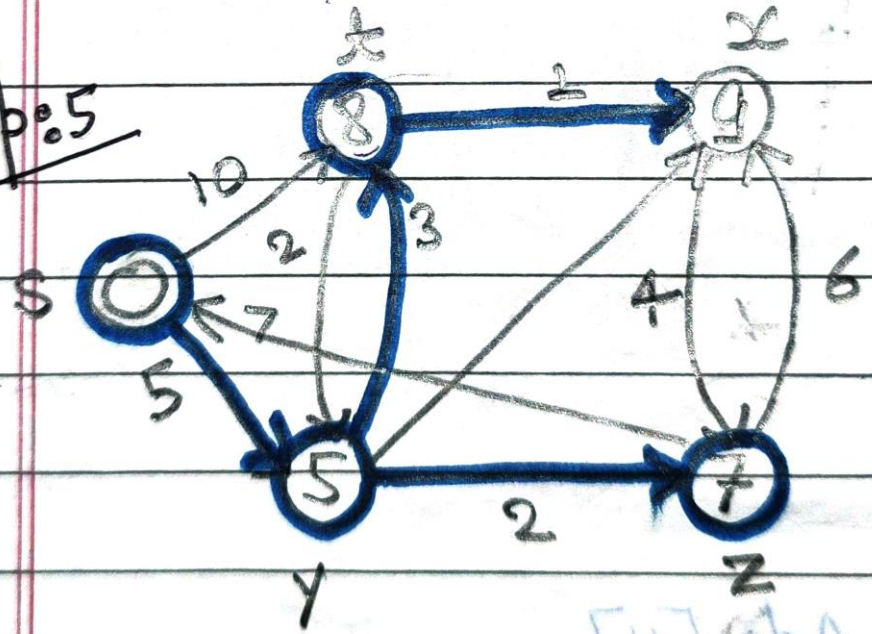
step:3



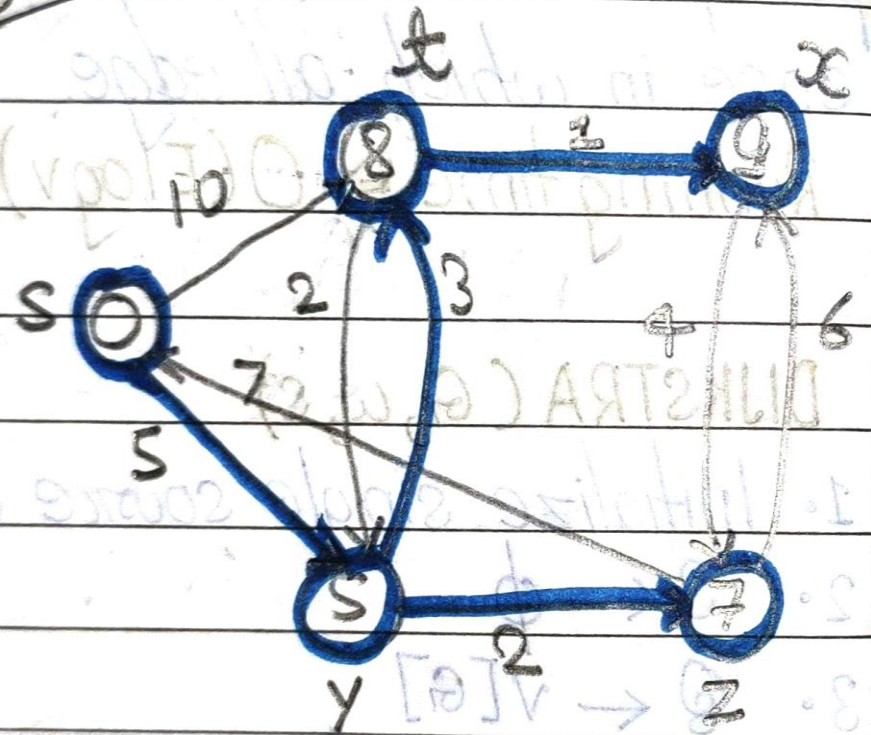
step: 4



step: 5



step: 6

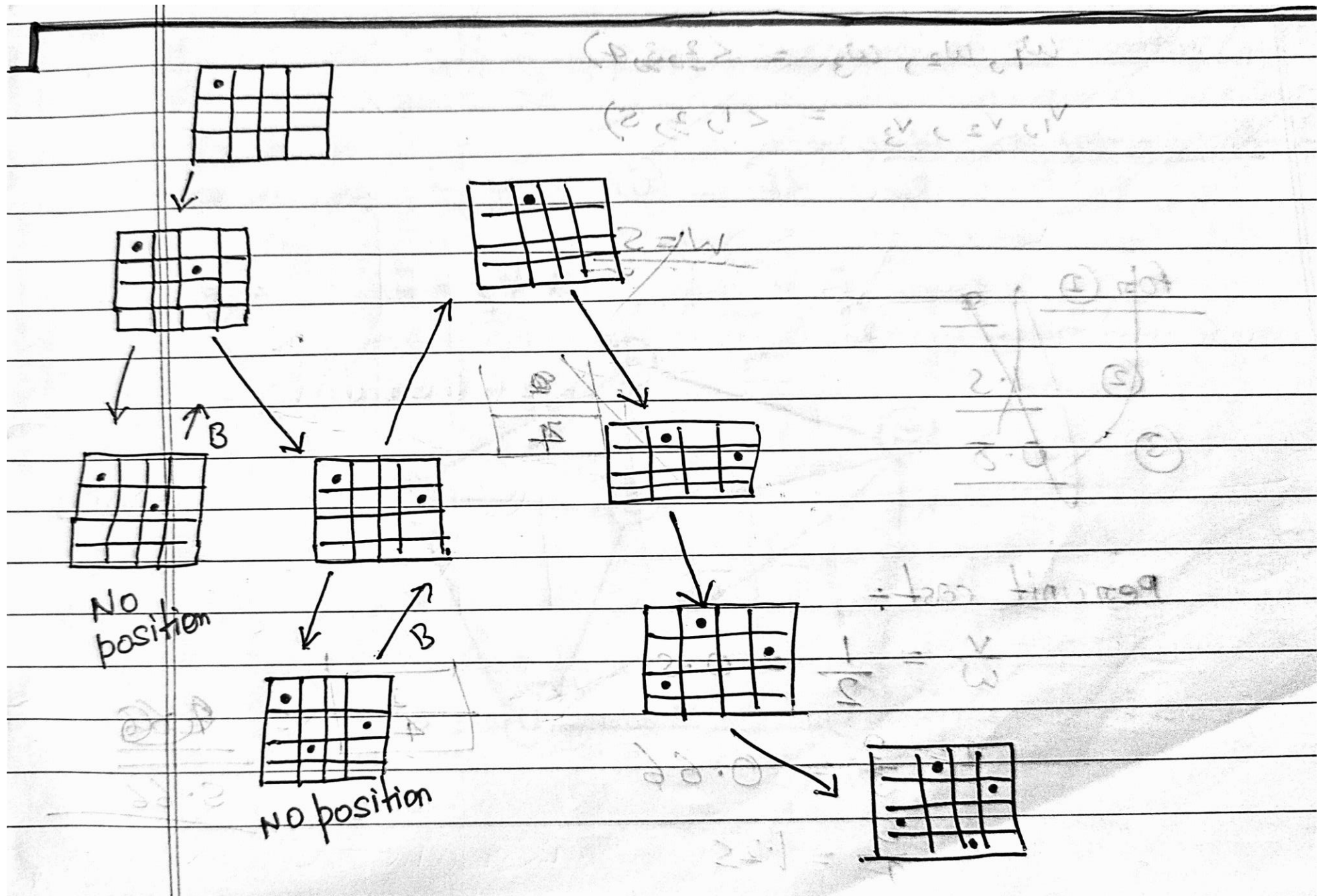


Backtracking Solution

- Backtracking is an effective approach to solve the N-Queens problem.
- The idea is to place queens one by one in different columns and check if they are in a valid position.
- If a valid placement is found, proceed to place the next queen, otherwise backtrack.

Pseudocode for Backtracking

- 1. Start by placing the first queen in the first column.
- 2. For each column, try placing the queen in a valid row.
- 3. If a queen can be placed, move to the next column and repeat.
- 4. If no valid position is found, backtrack to the previous column .



246135

	0				
			0		
					0
0					
		0			
				0	

6x6

24135

	0			
			0	
0				
		0		
				0

5x5

$(2,4) = \frac{1}{2}$

Time Complexity

- The time complexity of the N-Queens problem using backtracking is $O(N!)$.
- This is because for each queen, we need to check multiple positions and validate constraints.
- Efficient pruning techniques can help reduce the search space.

Visualization of Backtracking

- Backtracking explores all possible configurations of queen placements.
- When a conflict is detected (two queens threatening each other), the algorithm backtracks to the previous step.
- This process continues until all valid solutions are found.

Advantages and Challenges

- Advantages:
 - – Simple to implement.
 - – Guarantees a solution if one exists.
- Challenges:
 - – High time complexity ($O(N!)$).
 - – Not efficient for large N without optimizations.

A Hamiltonian cycle (or Hamiltonian circuit) is a concept from graph theory that refers to a cycle in a graph that visits each vertex exactly once and returns to the starting vertex. This is different from an Eulerian cycle, which requires visiting each edge exactly once.

Key Characteristics:

Cycle: It starts and ends at the same vertex.

Visits all vertices: Each vertex of the graph must be visited exactly once.

Undirected or Directed Graphs: Hamiltonian cycles can exist in both undirected and directed graphs.

Example: Consider a graph with vertices A, B, C, D:

- A possible Hamiltonian cycle would be: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Hamiltonian Path vs. Hamiltonian Cycle:

- A Hamiltonian path visits every vertex exactly once but doesn't necessarily return to the starting point.

- A Hamiltonian cycle is a closed path (it returns to the starting point).

Applications:

Traveling Salesman Problem (TSP): This is a famous problem in combinatorial optimization where one needs to find the shortest Hamiltonian cycle in a weighted graph.

Routing and Circuit Design: Hamiltonian cycles are used in designing efficient routes or circuits that need to pass through specific points exactly once.

Challenges:

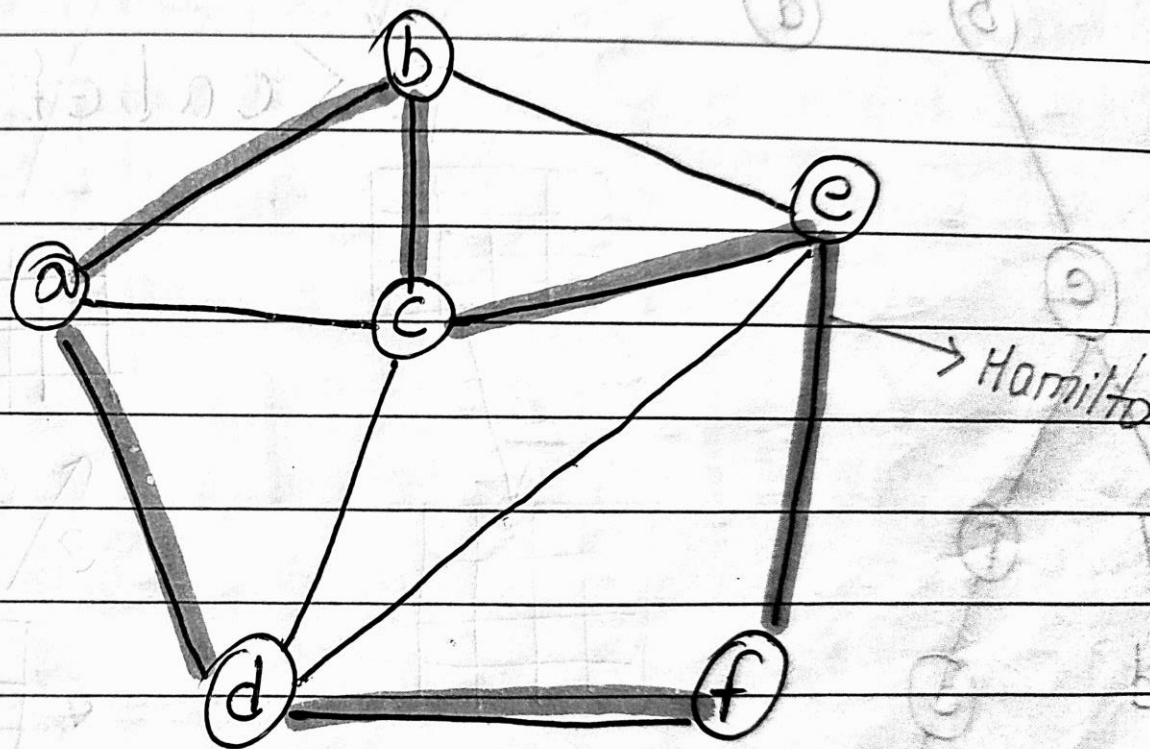
NP-Complete: Determining whether a Hamiltonian cycle exists in a given graph is NP-complete, meaning it's computationally difficult to solve for large graphs.

Would you like an example, or an algorithm related to finding Hamiltonian cycles?

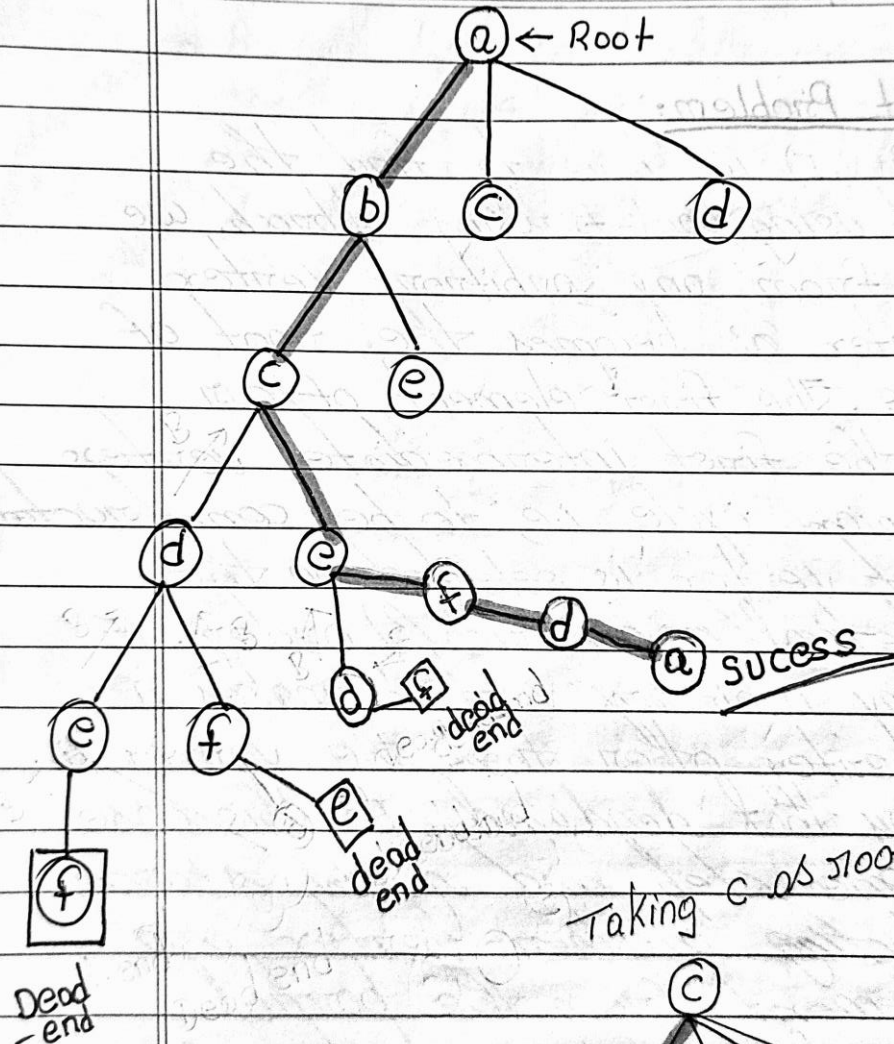
Hamiltonian Circuit Problem:

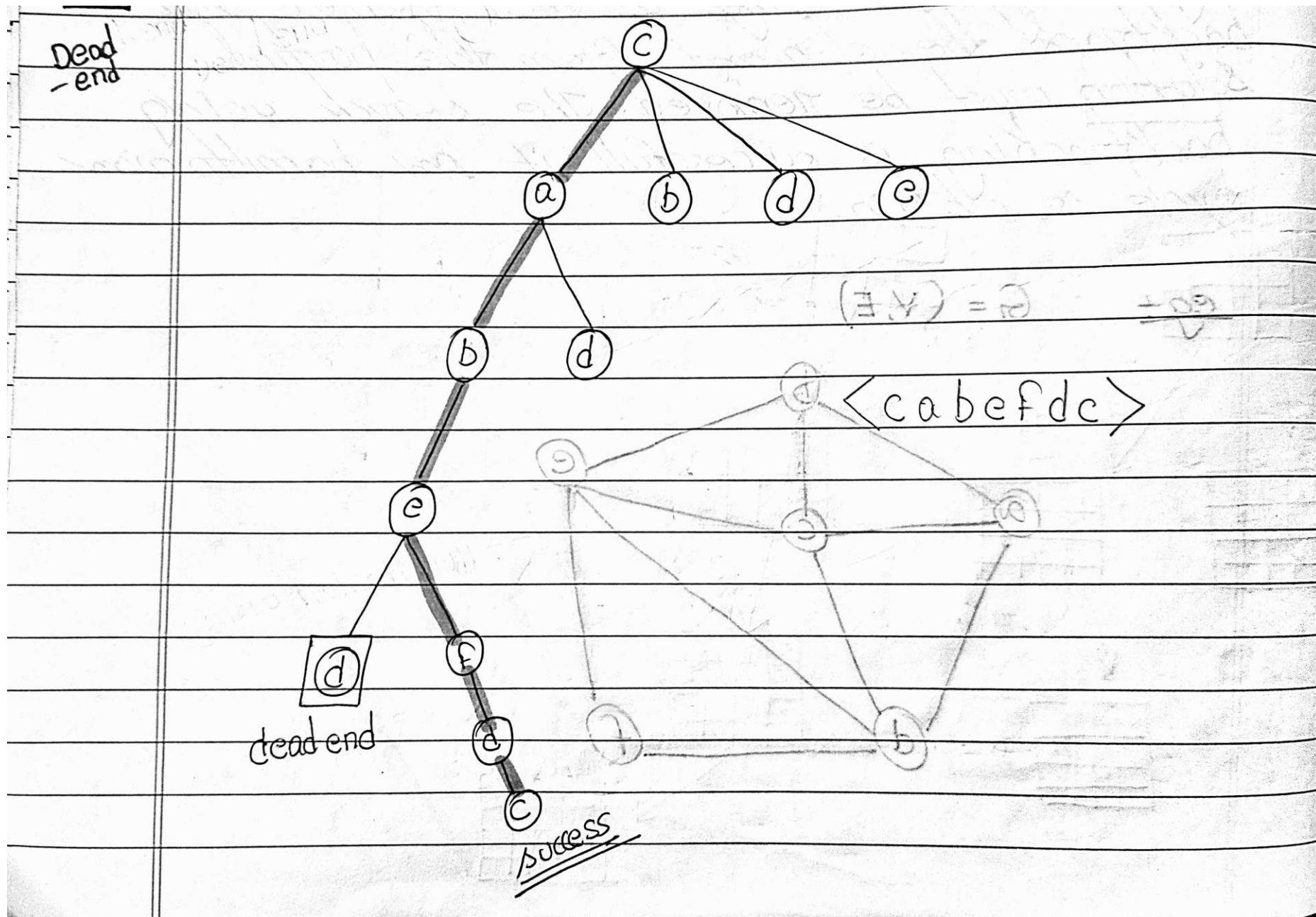
Given a graph $G(V, E)$ we have to find the hamiltonian circuit using back-tracking approach. We start our search from any arbitrary vertex say 'a'. This vertex 'a' becomes the root of our implicit tree. The first element of our partial solution is the first intermediate vertex of the hamiltonian cycle i.e. to be constructed. The next adjacent vertex is selected on the basis of alphabetical (or numerical) order. If at any stage any arbitrary vertex makes a cycle with any vertex other than the vertex 'a' then we say that deadend. In this case we backtrack one step and again search begins by selecting another vertex and backtrack the element from the partial solution must be removed. The search using backtracking is successful if an hamiltonian cycle is obtain.

eg: $G = (V, E)$

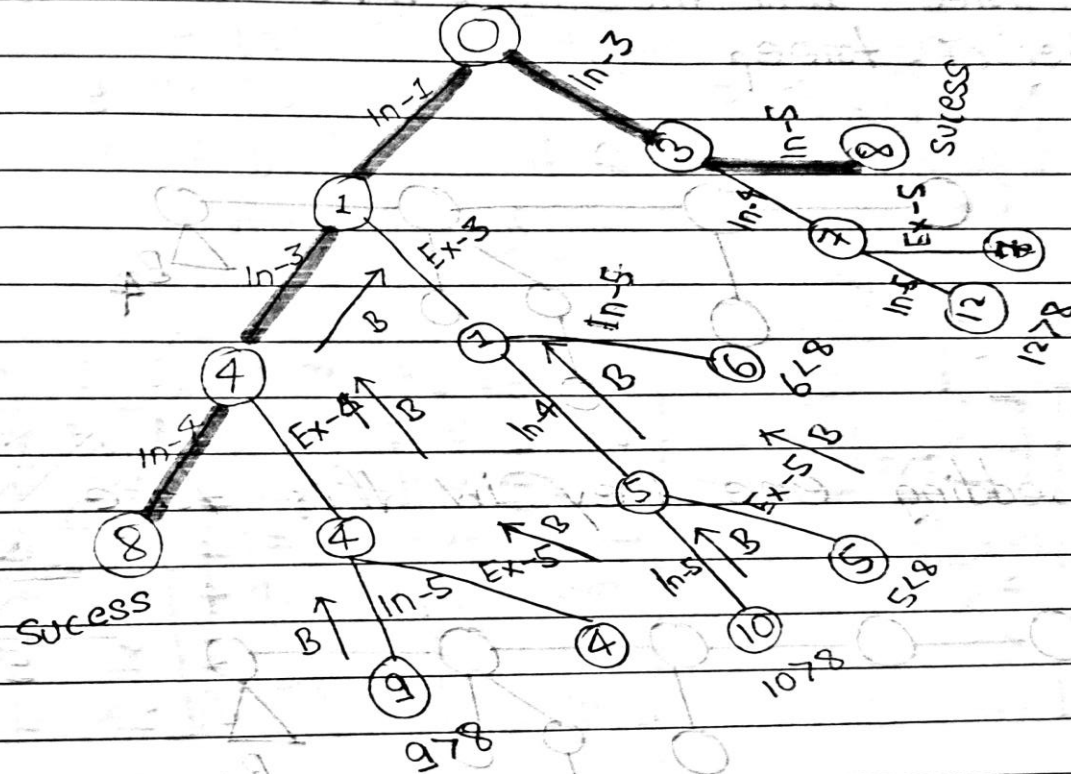


18-OCT-2013 (Friday)





Q → Given a set $S = \langle 1, 3, 4, 5 \rangle$ and $x = 8$ we have to find subset sum?



$$S = \langle 1, 3, 4, 5 \rangle$$

$$S_1' = \langle 1, 3, 4 \rangle$$

$S_2' = \langle 3, 5 \rangle$ Ans

Coloring Graphs

This handout:

- Coloring maps and graphs
- Chromatic number
- Applications of graph coloring

Coloring maps

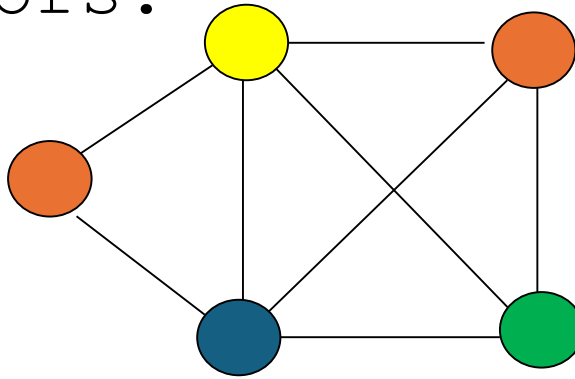
- Color a map such that two regions with a common border are assigned different colors.



- Each map can be represented by a graph:
 - Each region of the map is represented by a vertex;
 - Edges connect two vertices if the regions represented by these vertices have a common border.
- The resulting graph is called the *dual graph* of the map.

Coloring Graphs

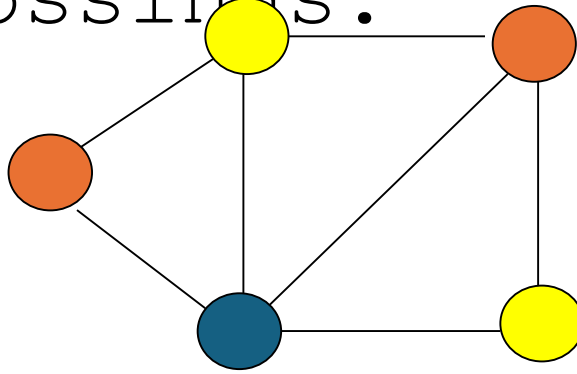
- **Definition:** A graph **has been colored** if a color has been assigned to each vertex in such a way that adjacent vertices have different colors.



- **Definition:** The **chromatic number** of a graph is the smallest number of colors with which it can be colored.
In the example above, the chromatic number is **4**.

Coloring Planar Graphs

- **Definition:** A graph is **planar** if it can be drawn in a plane without edge-crossings.



- **The four color theorem:** For every planar graph, the chromatic number is ≤ 4 .

*Was posed as a conjecture in the 1850s.
Finally proved in 1976 (Appel and Haken) by
the aid of computers.*

An application of graph coloring in scheduling

Twelve faculty members in a mathematics department serve on the following committees:

- *Undergraduate education*: Sineman, Limitson, Axiomus, Functionini
- *Graduate Education*: Graphian, Vectorades, Functionini, Infinitescu
- *Colloquium*: Lemmeau, Randomov, Proofizaki
- *Library*: Van Sum, Sineman, Lemmeau
- *Staffing*: Graphian, Randomov, Vectorades, Limitson
- *Promotion*: Vectorades, Van Sum, Parabolton

The committees must all meet during the first week of classes, but there are *only three time slots* available. Find a schedule that will allow all faculty members to attend the meetings of all committees on which they serve.

An application of graph coloring in exam scheduling

Suppose that in a particular quarter there are students taking each of the following combinations of courses:

- *Math, English, Biology, Chemistry*
- *Math, English, Computer Science, Geography*
- *Biology, Psychology, Geography, Spanish*
- *Biology, Computer Science, History, French*
- *English, Psychology, Computer Science, History*
- *Psychology, Chemistry, Computer Science, French*
- *Psychology, Geography, History, Spanish*

What is the minimum number of examination periods required for the exams in the ten courses specified so that students taking any of the given combinations of courses have no conflicts? Find a schedule that uses this minimum number of periods